Quarto 中的 R 代码与 RMarkdown 功能

1. Quarto 中使用 R 代码

在 Quarto 中，R 代码块的语法与 R Markdown 非常相似：

```
```{r}
# 示例 R 代码
summary(cars)
```
```

代码块由三个反引号 `` ``` 标识，并包含 {r}` 指明语言类型。

---

2. 常用代码块选项（Chunk Options）

Quarto 完全兼容 R Markdown 的代码块控制语法，同时也扩展了配置方式（YAML 风格）。

（1）执行与输出控制

选项	功能说明
`eval: true/false`	是否执行代码块
`echo: true/false`	是否显示代码
`output: asis / true / false`	控制输出显示方式
`warning: false`	隐藏警告信息
`message: false`	隐藏信息输出（如包加载提示）
`error: true/false`	是否显示代码错误
`include: false`	不在最终文档中显示（但仍执行）

示例：

```
```{r}
#| eval: true
#| echo: false
#| message: false

lm(mpg ~ wt, data = mtcars)
```
```

（2）输出格式与样式

选项	说明
`results: "markup" / "asis" / "hide"`	控制 R 输出文本的处理方式
`fig.cap:`	图表标题（caption）
`fig.width:` / `fig.height:`	图片尺寸（英寸）
`fig.align:`	图表对齐方式（`left`、`center`、`right`）

示例：

```
```{r}
#| fig.width: 6
#| fig.height: 4
#| fig.cap: "散点图示例"
plot(mtcars$wt, mtcars$mpg)
```
```

---

3. 参数化（Parameters）

Quarto 继承了 R Markdown 的参数化机制，可在 YAML 中定义参数并在 R 代码中引用。

```yaml
---
title: "参数化示例"
params:
  dataset: "mtcars"
  column: "mpg"
---
```

在代码中：

```r
data <- get(params$dataset)
summary(data[[params$column]])
```

可以通过命令行渲染不同参数：

```
quarto render report.qmd -P dataset="iris"
```

---

4. 缓存与依赖（Caching）

用于加速渲染、避免重复执行计算量大的代码。

```{r}
#| cache: true
#| dependson: "previous_chunk"
complex_model <- lm(mpg ~ wt + hp, data = mtcars)
```

- `cache: true` 表示缓存结果；
- `dependson` 指定此块依赖的上一个块，当依赖块更新时，此块会重新运行。

---

5. 代码执行上下文（context）

Quarto 可使用 `context:` 控制代码块在不同环境执行：

选项	说明
`context: local`	默认，本地执行
`context: setup`	仅初始化运行一次
`context: server`	用于 Shiny package 交互模式
`context: render`	仅在文档渲染时运行

---

6. 文档整体执行控制

在文档顶层 YAML 可以统一设置默认代码选项：

```yaml
---
title: "R 代码控制示例"
format: html
execute:
  echo: false
  warning: false
  message: false
  cache: true
---
```

这样所有代码块都会继承这些默认选项。

---

7. 交互式输出（R Widgets）

在 R 引擎中，可以直接插入支持 HTML 输出的交互包：

```r
library(plotly)
plot_ly(data = mtcars, x = ~wt, y = ~mpg, type = 'scatter', mode = 'markers')
```

兼容的包包括：

- `plotly`：交互式绘图
- `DT`：交互式表格
- `leaflet`：地图可视化
- `dygraphs`、`highcharter` 等

---

8. 超链接的添加（Adding Hyperlinks）

在 Quarto 或 Markdown 文档中，超链接（Hyperlink）用于实现页面跳转或访问外部网站。
它有两种主要用法：**内链（页内或站内）** 和 **外链（网页或文件）**。

---

1. 基本语法

Markdown 格式超链接的基本语法为：

```
[显示文字](链接地址)
```

示例：

```
请访问 [Quarto 官方网站](https://quarto.org) 了解更多信息。
```

渲染结果为：
👉 请访问 Quarto 官方网站 了解更多信息。

---

2. 页内链接（锚点跳转）

如果要链接到当前文档中的某个章节，可以使用该章节定义的 **锚点 ID**。
Quarto 为每个标题自动生成 ID，也可以手动设置：

示例：

```
## 数据准备 {#data}

[跳转到数据准备部分](#data)
```

点击文字即可跳转到标记 `{#data}` 的标题。

---

3. 跨页面链接

可以链接到网站、另一份 `.qmd` 文件，或该文件内的特定部分。

示例：

```
详见 [分析结果](results.qmd#summary)
```

这会跳转到 `results.qmd` 文件中 ID 为 `summary` 的位置。

4. 添加到按钮、图片等元素中

超链接也可以加在按钮或图片上（使用 HTML 语法）：

```
<a href="https://quarto.org" class="btn btn-primary">访问 Quarto 官网</a>
```

或：

```
[![Quarto 图标](quarto-logo.png)](https://quarto.org)
```

这会创建一个可点击图片链接。

---

5. 邮件或文件下载链接

```
[发送邮件](mailto:info@example.com)

[下载报告](files/report.pdf)
```

- `mailto:` 协议用于打开邮箱客户端；
- 普通路径则可指向同目录下的下载文件。

---

6. 提示框中添加超链接（融合示例）

```
::: {.callout-tip}
💡 你可以在提示框中添加链接，例如：
[查看折叠语法说明](#折叠块collapse-sections)
:::
```

小结

目标类型	示例语法	说明
外部网页	`[文本](https://example.com)`	打开外部站点
文件局部	`[文本](#id)`	跳转至当前页锚点
其他文档	`[文本](file.qmd#id)`	跳转到另一文档的章节
邮件链接	`[文本](mailto:someone@example.com)`	打开邮件客户端
文件下载	`[文本](path/to/file.pdf)`	下载或打开文件

Quarto 中的 YAML 配置（R 语言相关）

1. YAML 简介

在 Quarto 或 R Markdown 文档的开头使用三条短横线 `---` 包围的部分为 **YAML 头部（YAML front matter）**。它用于定义文档的元信息、输出格式、参数、执行环境等。

```yaml
---
title: "R 数据报告"
author: "分析团队"
date: "2026-04-28"
format: html
execute:
  echo: true
---
```

YAML 使用键值对语法，支持层级结构，缩进必须使用空格（通常为两个）。

---

2. 基本元信息字段

字段名	含义
`title`	文档标题
`subtitle`	副标题
`author`	作者（可以是单人或多人列表）
`date`	日期，支持 `"today"`
`format`	输出格式（如 `html`, `pdf`, `word`）
`toc`	是否生成目录（true / false）
`number-sections`	是否生成章节编号
`bibliography`	参考文献文件
`theme`	指定可视化主题

示例：

```yaml
---
title: "汽车数据分析"
subtitle: "基于 mtcars 的案例"
author:
  - name: Zhang Wei
  - name: Li Ming
date: today
format:
  html:
    toc: true
    toc-depth: 3
    theme: united
---
```

---

3. R 执行控制选项

在 YAML 中使用 `execute:` 段设置文档全局代码执行参数，相当于 R Markdown 中的 `knitr::opts_chunk$set()`。

选项	说明
`echo:`	是否显示源代码
`eval:`	是否执行代码
`warning:`	是否显示警告信息
`message:`	是否显示信息（如包加载消息）
`error:`	是否在错误时停止或显示错误
`cache:`	是否启用缓存
`freeze:`	是否重复使用已有输出（`auto` / `true` / `false`）
`include:`	是否将输出包含在文档中
`output:`	控制文字输出方式（如 `asis`）

示例：

```yaml
---
execute:
  echo: false
  warning: false
  message: false
  cache: true
  freeze: auto
---
```

---

4. 参数化报告（Parameters）

用于生成可复用的动态报告。

```yaml
---
title: "参数化分析报告"
params:
  dataset: "mtcars"
  variable: "mpg"
  threshold: 20
---
```

在 R 代码块中调用参数：

```r
data <- get(params$dataset)
subset(data, data[[params$variable]] > params$threshold)
```

命令行渲染示例：

```bash
quarto render report.qmd -P dataset="iris" -P variable="Sepal.Length"
```

---

5. 输出格式控制（format）

HTML 输出示例

```yaml
format:
  html:
    toc: true
    toc-depth: 3
    number-sections: true
    code-fold: true
    code-summary: "显示代码"
    embed-resources: true
    theme: cosmo
    df-print: paged
```

常用参数：

参数	说明
`code-fold`	显示/隐藏代码的折叠按钮
`df-print`	数据框打印方式：`default`、`paged`、`kable`
`embed-resources`	内嵌外部资源到 HTML 文件
`code-tools`	是否显示“复制代码”等工具按钮

PDF 输出示例

```yaml
format:
  pdf:
    documentclass: article
    cite-method: biblatex
    keep-tex: true
```

Word 输出示例

```yaml
format:
  docx:
    reference-doc: "template.docx"
```

该选项可用于自定义 Word 样式模板。

---

6. Shiny 支持（R 特有）

启用 Shiny 交互运行环境，只需在 YAML 中添加 `server: shiny`：

```yaml
---
title: "Shiny 交互示例"
server: shiny
format: html
---
```

然后即可在文档中使用 Shiny 输入、输出与反应式表达式。

---

7. 资源管理（Resources）

可以通过 `resources:` 声明外部文件（图片、样式、脚本等）：

```yaml
---
resources:
  - "images/logo.png"
  - "styles/main.css"
  - "scripts/interact.js"
---
```

---

8. 常见 YAML 区块总结

分类	区块名称	常用选项
元信息	`title`, `author`, `date`	设置标题、作者、日期等

分类	区块名称	常用选项
执行设置	`execute:`	控制 R 代码全局执行行为
参数设置	`params:`	定义参数化输入
输出格式	`format:`	控制 HTML、PDF、Word 等格式
资源管理	`resources:`	添加外部文件
Shiny 支持	`server: shiny`	启动交互式环境

Quarto 页面布局（Page Layout）

1. 页面布局的组成

Quarto 文档的页面布局主要由以下几个部分组成：

1. **页眉（Header）与页脚（Footer）**
2. **封面（Title Page）**
3. **页边距（Margins）与纸张尺寸（Page Size）**
4. **页码与章节编号（Page Numbers & Sections）**
5. **列布局（Columns）与宽度控制**
6. **HTML 页面外观布局（主题、侧边栏、导航栏）**

不同输出格式（HTML、PDF、Docx）在对应布局支持上略有区别。

---

2. HTML 页面布局

HTML 格式在 Quarto 中支持更多 **网页式布局控制**，通过 YAML 的 `format.html:` 段落配置。

示例：

```yaml
---
title: "R 数据分析报告"
format:
  html:
    toc: true
    toc-location: left
    toc-depth: 3
    number-sections: true
    theme: cosmo
    css: styles.css
    page-layout: full
---
```

HTML 布局选项

选项	说明
`page-layout:`	控制网页布局类型
`theme:`	主题样式（如 `default`，`cosmo`，`flatly` 等）
`toc:`	开启目录（Table of Contents）
`toc-location:`	目录位置，可选 `left`、`right`、`floating`
`toc-expand:`	初始展开层级
`sidebar:`	添加侧边栏导航
`navbar:`	顶部导航栏配置
`css:`	附加自定义样式

page-layout 的常见值

值	说明
`default`	默认布局（带页边距）
`full`	页面宽度铺满
`article`	适合论文与报告
`centered`	居中、居窄布局
`sidebar`	页面左/右包含导航栏

示例（带侧边栏）：

```yaml
---
format:
  html:
    page-layout: sidebar
    sidebar:
      contents:
        - section: "数据准备"
          contents: ["intro.qmd", "clean.qmd"]
        - section: "分析结果"
          contents: ["model.qmd", "visual.qmd"]
---
```

---

3. PDF 布局设置

PDF 输出使用 LaTeX 模板控制页面布局，通过 YAML 的 `format.pdf:` 段落定义。

基本设置示例

```yaml
---
format:
  pdf:
    documentclass: article
    papersize: a4
    geometry: "margin=2.5cm"
    fontsize: 11pt
    linestretch: 1.3
    number-sections: true
    toc: true
---
```

PDF 布局关键参数

参数	说明
`documentclass:`	LaTeX 文档类型（如 article、report、book）
`papersize:`	页面尺寸（`a4`，`letter` 等）
`geometry:`	直接传递到 `geometry` 宏包，可定义边距
`fontsize:`	主体字体大小（10pt、11pt、12pt）
`linestretch:`	行距倍数
`header-includes:`	在导言区插入自定义 LaTeX 代码
`include-before-body:` / `include-after-body:`	在正文前后插入内容
`page-numbering:`	设置页码样式（arabic、roman、none）

页眉页脚示例

```yaml
---
format:
  pdf:
    header-includes: |
      \usepackage{fancyhdr}
      \pagestyle{fancy}
      \fancyhead[L]{R 数据报告}
      \fancyhead[R]{分析结果}
      \fancyfoot[C]{\thepage}
---
```

---

4. Word 布局设置

Word 输出主要依赖模板文档（`reference-doc:`），可以自定义样式与页边距。

```yaml
---
format:
  docx:
    reference-doc: "template.docx"
    toc: true
    number-sections: true
---
```

通过 Word 模板，控制：

- 页眉页脚位置与样式；
- 章节标题样式；
- 页边距与字体；
- 段间距与编号格式。

---

5. 多栏布局与宽度控制

Quarto 可以在文档中使用多列排版（类似 LaTeX 的 `multicol` 环境）。

简单多栏布局

```
::: columns
::: column
左侧内容
:::

::: column
右侧内容
:::
:::
```

控制列宽比例

```
::: columns
::: column
:width: 40%
左边列
:::

::: column
:width: 60%
右边列
:::
:::
```

---

6. 页面标题、目录与章节编号

在 YAML 中控制：

```yaml
---
title: "数据报告"
subtitle: "分析与结果"
number-sections: true
toc: true
toc-depth: 3
---
```

- `toc: true` 开启目录；
- `toc-depth:` 设置目录层级；
- `number-sections: true` 自动为章节编号；
- 可以结合 `number-depth:` 限制编号层级。

---

7. 页眉与页脚（HTML 输出）

HTML 中可通过自定义 CSS 或 template 配置页眉和页脚。例如：

```yaml
---
format:
  html:
    include-before-body: header.html
    include-after-body: footer.html
---
```

`header.html` 中可以放置 Logo、导航、日期等内容。

---

8. 示例：完整布局配置

```yaml
---
title: "R 数据分析报告"
author: "分析小组"
date: today
format:
  html:
    theme: simplex
    page-layout: sidebar
    toc: true
    toc-depth: 3
    toc-location: left
    code-fold: true
    include-after-body: footer.html
  pdf:
    papersize: a4
```

```
    geometry: "margin=2.5cm"
    fontsize: 11pt
    number-sections: true
    toc: true
execute:
  echo: false
  warning: false
---
```

Quarto 页面跳转与交互布局功能

1. 面内跳转（Anchors & Links）

1. 自动章节锚点

Quarto 会自动为每个标题生成锚点（anchor）。
可以通过 Markdown 语法在文档中跳转到指定标题。

示例：

```
## 数据准备 {#data}

一些描述性文字...

可以在别处引用该小节：

详见 [数据准备](#data)。
```

说明：

- `{#data}` 是手动定义锚点 ID；
- `[数据准备](#data)` 是页面内跳转。

---

2. 自定义跳转按钮

你也可以用 HTML 语法生成按钮样式跳转：

```
<a href="#summary" class="btn btn-primary">跳到总结部分</a>
```

`class="btn btn-primary"` 利用 Bootstrap 样式生成按钮。

---

3. 外部文件跳转

```
详见 [完整报告](results.qmd#model)。
```

这会跳转到另一个 `.qmd` 文件中的特定锚点（ID 为 `model`）。

---

2. Tabset（选项卡布局）

Quarto 提供一种简洁方式来创建选项卡组，即 `::: {.panel-tabset}`。

这种语法常用于同一部分内容展示多种视图，例如代码与图表、不同模型结果等。

基本语法：

```
::: {.panel-tabset}
### 模型概览
这里展示模型背景与基本信息。

### 回归结果
展示回归系数表格。

### 可视化图形
展示结果图表。
:::
```

渲染后会生成一个带标签页的内容区域。

自定义标签页顺序与命名

- Tab 的标题来自于其下一级标题；
- 可以通过在标题后添加 `{#id}` 定义链接目标；
- YAML 中支持全局启用 tabset，但推荐局部使用。

嵌套使用

`tabset` 可以嵌套使用：

```
::: {.panel-tabset}
### 数据分析
#### 概览
#### 统计
#### 可视化
:::
```

嵌套时建议保持层级清晰，否则会出现渲染混乱。

---

3 . 折叠块（Collapsible Sections）

Quarto 提供可折叠内容块，用于隐藏次要内容，例如详细代码或附录。

语法示例：

```
::: {.callout-note collapse="true"}
#### 附录：数据说明

该部分为附加信息，默认折叠。
:::
```

参数说明：

参数	作用
`collapse="true"`	默认折叠
`collapse="false"`	默认展开
`type`	设置提示样式（`note`、`tip`、`warning`、`important` 等）

---

4. 面板与列布局（Panels & Columns）

在 Quarto 中，可以用 `panel` 来并排放置图、表或说明内容。

示例：两列并排展示

```
::: columns
::: {.column width="50%"}
这里放模型代码或参数说明。
:::

::: {.column width="50%"}
这里展示绘图结果。
:::
:::
```

输出时就是左右两栏布局。

示例：图片与文字并排

```
::: {.columns}
::: {.column width="60%"}
![散点图](scatter.png)
:::

::: {.column width="40%"}
**分析说明:**
图中显示变量之间的线性关系明显。
:::
:::
```

5. Callout / 提示框布局

`:::{.callout-类型}` 用来强调某种内容，常见类型如下：

类型	样式
`.callout-note`	普通说明
`.callout-tip`	提示信息
`.callout-warning`	警告
`.callout-important`	重要内容
`.callout-caution`	注意事项

示例：

```
::: {.callout-warning}
**警告:** 数据量过大时，绘图性能可能受影响。
:::
```

可与 `collapse="true"` 结合使用。

---

6. 导航与页内结构增强

在大型报告中，除了标题跳转，还可以配置 **侧边目录（sidebar）** 和 **导航栏（navbar）**。

YAML 中配置侧边栏导航示例

```yaml
---
format:
  html:
    page-layout: sidebar
    sidebar:
      title: "数据分析报告"
      style: "floating"
      contents:
        - section: "数据清洗"
          contents:
            - clean.qmd
            - transform.qmd
        - section: "建模与结果"
          contents:
            - model.qmd
            - visualize.qmd
---
```

这样会在左侧生成可点击跳转的目录。

7. 实用组合示例

```
## 模型结果 {#model-results}

::: {.panel-tabset}
### 概览
模型共使用五个自变量。AIC=1023.4。

### 系数表
| 变量 | 系数 | p值 |
|------|------|-----|
| mpg  | -0.3 | 0.002 |
| hp   | 0.08 | 0.01  |

### 可视化
![](plot.png)
:::

[跳转到结论部分](#conclusion)
```

配合上方的锚点跳转，这个结构既有交互选项卡，又能快速定位到其他部分，提高报告的阅读体验。

---

8. 总结

功能	语法/关键字	用途
页面跳转	`[文本](#id)`	页面内定位
Tabset	`::: {.panel-tabset}`	多标签切换
折叠块	`collapse="true"`	展开/折叠内容
Callout	`.callout-note / tip / warning`	高亮提示区域
Columns	`::: columns`	多列布局
Sidebar	`page-layout: sidebar`	页面导航栏

太好了！现在我们进入 **Quarto 中的 Shiny 交互组件（Shiny Widgets）** 部分。
这部分内容让你的报告不仅是静态展示，还能实现 **即时交互分析** —— 用户可以通过输入控件、滑块、选择器等组件实时改变分析结果或图形。

---

1. Quarto 与 Shiny 的整合方式

Quarto 能将 R 的 **Shiny 框架** 直接嵌入文档中，通过在 YAML 中启用：

```yaml
---
title: "交互式数据分析报告"
server: shiny
format: html
execute:
  echo: false
---
```

`server: shiny` 表示该页面将在 **Shiny runtime** 下运行。
在这种模式中：

- 每次修改输入组件的值时，会 **重新执行相关 R 代码块**；
- 输出（如表格、图形）可以实时更新。

---

2. Shiny 输入组件（Input Widgets）

Shiny 输入组件用于接收用户输入，并可在 R 代码中通过 `input$xxx` 获取其值。

函数	控件类型	示例
`sliderInput()`	滑块	选择数值范围
`selectInput()`	下拉选择框	从选项中选择
`radioButtons()`	单选按钮组	选择一项
`checkboxInput()`	单个复选框	开启 / 关闭
`checkboxGroupInput()`	多选框组	选择多项
`numericInput()`	数字输入框	输入数值
`textInput()`	文本输入框	输入字符串
`dateInput()`	日期选择器	选择日期
`actionButton()`	动作按钮	点击触发行为

示例 1：滑块控制图形范围

```
```{r}
#| context: server
sliderInput("bins", "选择直方图分箱数:", 5, 50, 20)
```

```{r}
hist(faithful$eruptions, breaks = input$bins, col = "skyblue")
```
```

说明：

- `context: server` 指定该代码块运行在 Shiny 上下文；
- `input$bins` 即滑块选定的值；
- 图表会随滑块移动而更新。

示例 2：选择器切换数据集

```
```{r}
# 选择不同内置数据
selectInput("dataset", "选择数据集:", choices = c("mtcars", "iris", "PlantGrowth"))
```

```{r}
# 根据选择显示数据
dataset <- get(input$dataset)
head(dataset)
```
```

用户选择不同数据集后，`input$dataset` 的值改变，数据表随之更新。

---

3. Shiny 输出组件（Render Functions）

配合输入组件，我们使用 **Shiny 渲染函数** 来输出交互结果。

函数	输出类型	描述
`renderPlot()`	图形	绘制动态图
`renderTable()`	表格	输出数据表
`renderText()`	文本	输出字符串
`renderPrint()`	打印结果	
`renderPlotly()`	交互式 Plotly 图	
`renderDataTable()`	交互式表格（DT 包）	

对应 HTML 输出标签分别为：

输出标签函数	渲染函数配对
`plotOutput("id")`	`renderPlot({ ... })`
`textOutput("id")`	`renderText({ ... })`
`tableOutput("id")`	`renderTable({ ... })`

输出标签函数	渲染函数配对
`dataTableOutput("id")`	`renderDataTable({ ... })`

示例 3：输入 + 输出结合

```
```{r}
#| context: server
sliderInput("n", "样本量:", 10, 100, 30)
```

```{r}
renderPlot({
  hist(rnorm(input$n), col = "orange", main = paste("n =", input$n))
})
```
```

移动滑块时，`rnorm(input$n)` 的样本量实时变动，直方图刷新。

---

4. 布局与联动（Layout & Reactivity）

你可以使用 **Shiny 布局函数** 或 **Quarto 的布局语法** 组织交互组件。

方式 1：传统 Shiny 布局

```
```{r}
fluidRow(
  column(4, sliderInput("bins", "分箱数", 1, 50, 20)),
  column(8, plotOutput("histPlot"))
)

renderPlot({
  hist(faithful$eruptions, breaks = input$bins)
})
```
```

方式 2：Quarto 多列布局语法

```
::: columns
::: {.column width="30%"}
```{r}
sliderInput("bins", "分箱数:", 1, 50, 20)
```
:::

::: {.column width="70%"}
```{r}
```
```

```
renderPlot({
  hist(faithful$eruptions, breaks = input$bins)
})
```
:::
:::

---

5. 反应式表达式（Reactive Expressions）

当多个输出依赖同一运算时，可避免重复计算，用 `reactive()` 定义共享变量。

```{r}
#| context: server
sliderInput("bins", "分箱数:", 5, 50, 20)

data_reactive <- reactive({
  hist(faithful$eruptions, breaks = input$bins, plot = FALSE)
})

renderPlot({
  plot(data_reactive(), col = "lightblue")
})
```

---

6. 进阶：交互式可视化与表格

1. 使用 Plotly 增强交互

```{r}
#| context: server
sliderInput("n", "点数量:", 10, 200, 50)
```

```{r}
renderPlotly({
  library(plotly)
  df <- data.frame(x = rnorm(input$n), y = rnorm(input$n))
  plot_ly(df, x = ~x, y = ~y, type = 'scatter', mode = 'markers')
})
```

2. 使用 DataTable 交互展示数据

```
```{r}
selectInput("data", "选择数据集:", c("iris", "mtcars"))
```

```{r}
renderDataTable({
  get(input$data)
})
```
```

生成的表格支持 **分页、搜索、排序。**

---

7. 按钮控制交互（Actions）

`actionButton()` 提供触发行为，而不是持续响应。

例如：

```
```{r}
actionButton("go", "运行分析")
```

```{r}
observeEvent(input$go, {
  showNotification("分析已完成！", type = "message")
})
```
```

---

8. 绑定 `panel-tabset` 与 Shiny

你可以将不同的交互分析放入不同 Tab 页中。

```
::: {.panel-tabset}
### 数据选择
```{r}
selectInput("ds", "选择数据集", c("mtcars", "iris"))
```

可视化
```{r}
renderPlot({
  data <- get(input$ds)
  plot(data)
})
```
:::
```

用户在第一个 Tab 选择数据集，第二个 Tab 的图表同步更新。

---

9. 完整示例

```
---
title: "Shiny 交互式分析报告"
server: shiny
format: html
execute:
  echo: false
---
```

```
## 动态直方图

::: columns
::: {.column width="30%"}
```{r}
sliderInput("bins", "分箱数:", min = 5, max = 50, value = 20)
:::
::: {.column width="70%"}
```{r}
renderPlot({
  hist(faithful$eruptions, breaks = input$bins, col = "steelblue",
       main = paste("分箱数 =", input$bins))
})
:::
:::
```

这就是一个可实时调整直方图的报告，运行时自动实现交互式刷新。

---

10. 小结

分类	函数 / 功能	说明
启用 Shiny	`server: shiny`	打开交互模式
输入组件	`sliderInput()`, `selectInput()` 等	获取用户输入
输出组件	`renderPlot()`, `renderText()` 等	输出动态结果
反应式表达式	`reactive()`, `observeEvent()`	控制刷新逻辑
布局	`columns`, `panel-tabset`	页面结构组织
高级交互	`Plotly`, `DT::renderDataTable()`	高级可视化与表格